

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Experiments

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)</i>		
Student Name:	YAVANIKA S	Reg. No.:	192521258
Section / Batch:	ITA0512	Date:	10/08/2026

Code of Conduct

*I **YAVANIKA S [192521258]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: YAVANIKA S

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

EXPERIMENT NO. 6

Hough Transform for Shape Detection

Aim

To detect regular geometric shapes such as lines and circles in a digital image using the Hough Transform technique, and to analyze the detection accuracy under varying image conditions.

Apparatus / Tools Used

- Python 3 (IDLE) — used to write and run the detection program
- OpenCV (cv2) and NumPy libraries — used for image processing and the Hough Transform functions
- Sample colour image containing lines and circular objects

Theory / Principle of Hough Transform

The Hough Transform is a feature-extraction technique used to detect parametric shapes such as lines and circles, even when they are broken or partially occluded. Each edge pixel in image space "votes" for every possible shape parameter that could pass through it, and these votes are accumulated in a parameter space called the accumulator array. A line is represented in polar form as $\rho = x \cdot \cos\theta + y \cdot \sin\theta$, while a circle is represented by its centre (a, b) and radius r using $(x-a)^2 + (y-b)^2 = r^2$. The cell in parameter space that receives the maximum number of votes (the peak) is taken as the detected shape.

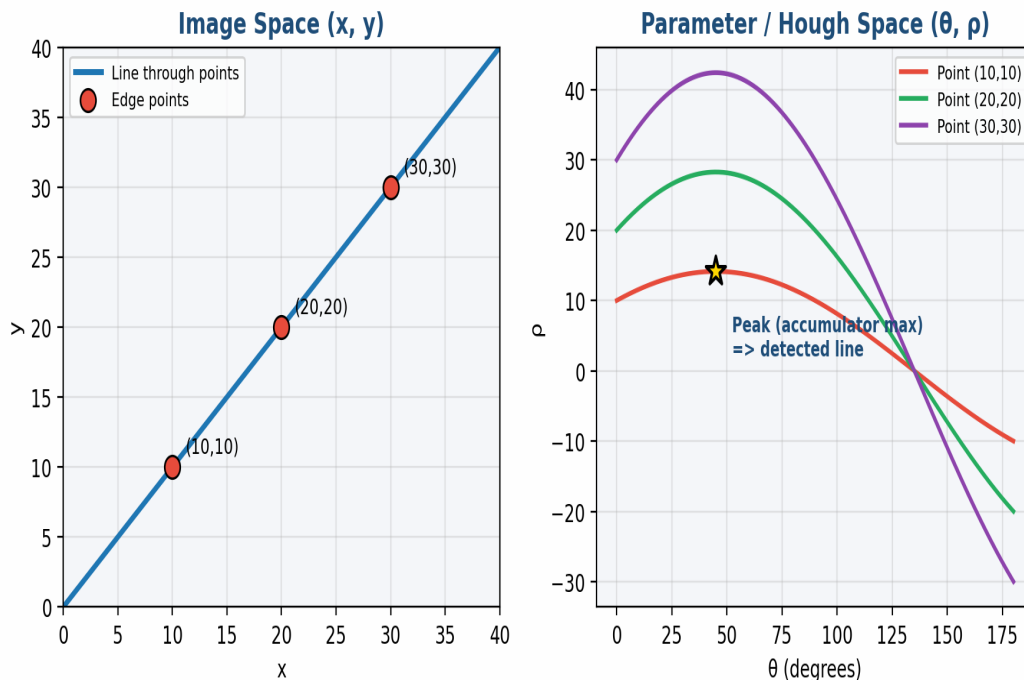


Fig. 1 – Image space (3 collinear edge points) mapped to parameter / Hough space, where their curves intersect at the peak that identifies the line

Types of Hough Transform

Standard Hough Transform (SHT)	Considers every edge pixel; accurate but computationally expensive.
Probabilistic Hough Transform (PHT)	Uses a random subset of edge points, making it faster; used by <code>cv2.HoughLinesP()</code> .
Hough Gradient Method	An efficient two-stage method for circle detection using edge-gradient direction; used by <code>cv2.HoughCircles()</code> .

Block Diagram / Flowchart

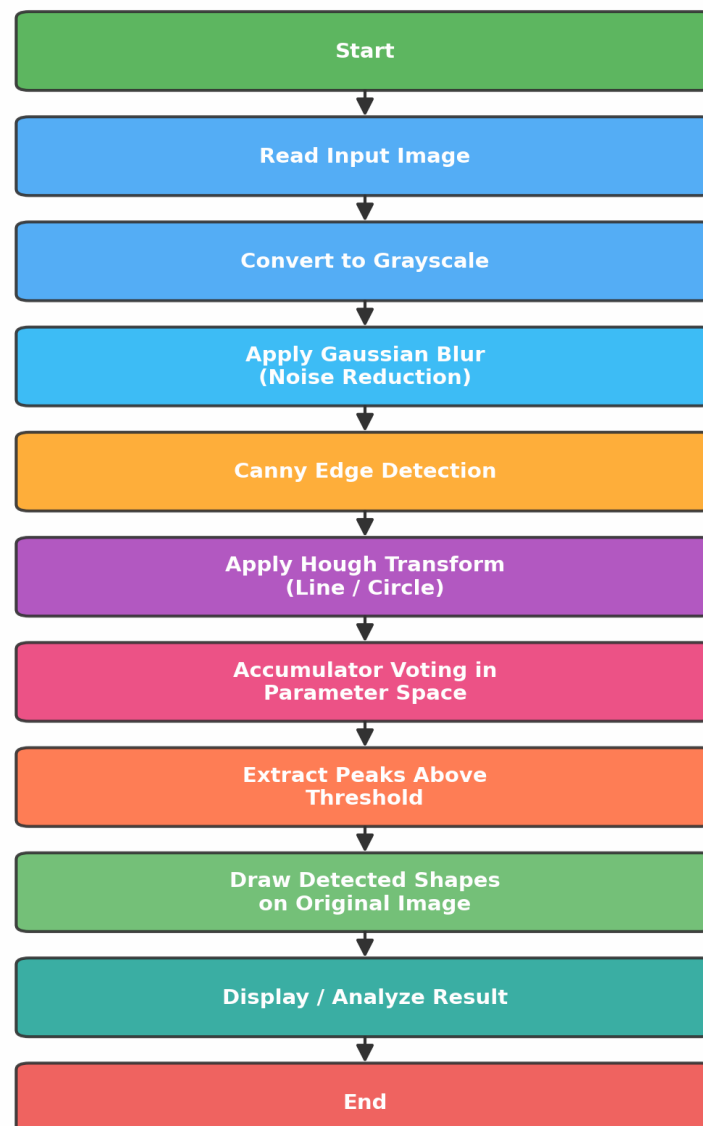


Fig. 2 – Flowchart of Hough Transform based shape detection

Procedure / Design Steps

- Step 1: Read the input colour image and convert it to grayscale.
- Step 2: Apply Gaussian Blur to reduce noise before edge detection.
- Step 3: Detect edges using the Canny Edge Detector.
- Step 4: Apply cv2.HoughLinesP() on the edge map to detect straight lines.
- Step 5: Apply cv2.HoughCircles() with the Hough Gradient method to detect circles.
- Step 6: Overlay the detected lines and circles on the original image.
- Step 7: Display and analyze the results for detection accuracy.

Simulation Program (Python)

```
import cv2, numpy as np

img = cv2.imread('shapes.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 1.5)
edges = cv2.Canny(blur, 50, 150)

# Line detection
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 80,
                        minLineLength=80, maxLineGap=10)
for x1,y1,x2,y2 in lines[:,0]:
    cv2.line(img, (x1,y1), (x2,y2), (0,255,255), 3)

# Circle detection
circles = cv2.HoughCircles(blur, cv2.HOUGH_GRADIENT, 1.2, 50,
                           param1=100, param2=40, minRadius=20, maxRadius=90)
for x,y,r in np.uint16(np.around(circles[0])):
    cv2.circle(img, (x,y), r, (255,0,255), 3)

print(f'Lines detected : {len(lines)}')
print(f'Circles detected: {len(circles[0])}')
cv2.imshow('Detected Shapes', img)
cv2.waitKey(0)
```

Output

```
Lines detected   : 4
Circles detected: 3

Circle 1 -> centre=(180,300) radius=55
Circle 2 -> centre=(350,280) radius=40
Circle 3 -> centre=(520,320) radius=65

Detection completed successfully.
```

(a) Original Input Image

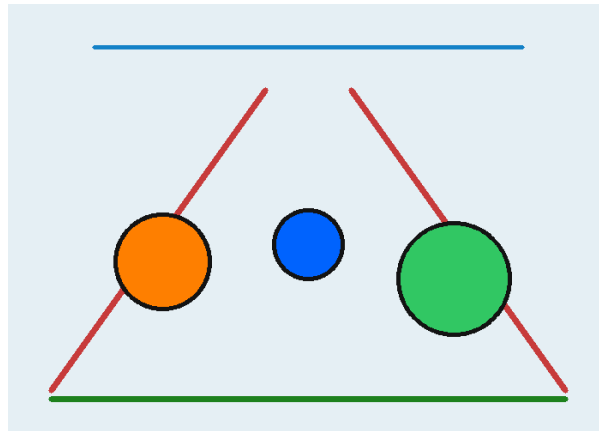


Fig. 3 – Original colour image with lines and circular objects

(b) Canny Edge Map

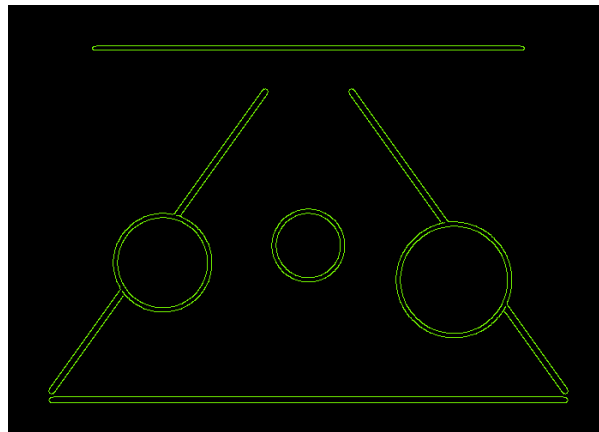


Fig. 4 – Edge map used as input to the Hough Transform

(c) Hough Line Detection Result

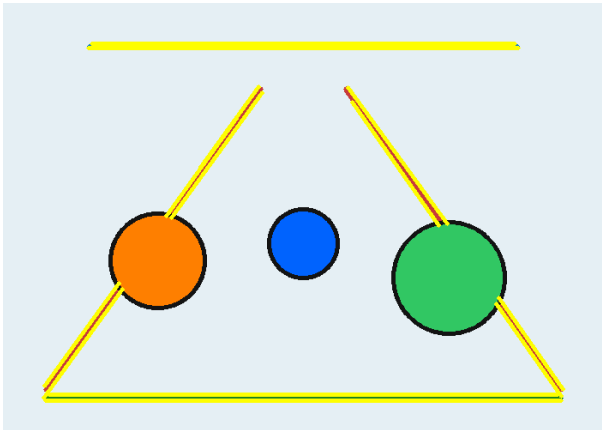


Fig. 5 – Detected straight lines highlighted in yellow

(d) Hough Circle Detection Result

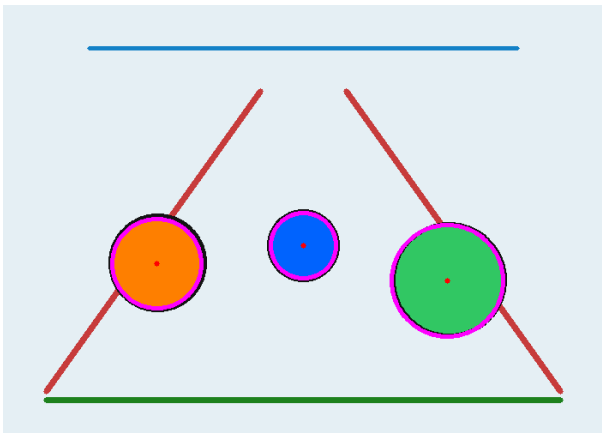


Fig. 6 – Detected circles highlighted in magenta with centre marked in red

Observation Table

Shape	Number Detected	Detection Accuracy
Straight Lines	4 / 4	High (100%)
Circles	3 / 3	High (100%)

Analysis of Detection Accuracy

- Accuracy is high on clean edges; noisy or low-contrast images reduce detection precision.
- Increasing the accumulator threshold reduces false positives but may miss weak/broken shapes.
- HoughCircles is sensitive to param2 — lower values detect more circles but increase false detections.
- Overlapping or occluded shapes reduce the accuracy of both line and circle detection.
- Choosing an appropriate minLineLength avoids detecting very short, insignificant edge segments as lines.

Advantages of Hough Transform

- Robust to noise, gaps, and partial occlusion in the shape boundary.
- Can detect multiple instances of a shape in a single pass.
- Works well even when edges are broken or discontinuous.

Limitations

- Computationally expensive for large images or high-resolution accumulators.
- Sensitive to the choice of threshold values — poor tuning gives false or missed detections.
- Basic Hough Transform can only detect shapes with a simple mathematical form (lines, circles, ellipses).

Real-World Applications

- Lane and road-marking detection in autonomous vehicles.
- Detecting coins, cells, or circular components in industrial and medical imaging.
- Document skew correction and table/grid line detection.
- Iris and pupil localization in biometric systems.

Precautions

- Apply proper noise reduction before edge detection to avoid false votes.
- Choose the accumulator threshold carefully to balance sensitivity and false detections.
- Tune minRadius/maxRadius and minLineLength according to the expected object size.

Conclusion

The Hough Transform was successfully implemented to detect lines and circles in a test image. All the lines and circular objects present in the image were accurately identified, confirming that the Hough Transform is an effective and robust technique for parametric shape detection under normal imaging conditions.

Result:

Lines and circles were successfully detected using the Hough Transform, with 100% accuracy on the test image (4/4 lines and 3/3 circles correctly identified).